Atelier_1 : Gestion Commerciale

Rapport de travaux pratiques : SQL / MySQL

Base de données : **atelier_commerce**

Nom et prénom : EL HARTI OUMAIMA

Module : M203

Enseignant : Karim Azeggouar

Date : 10/11/2026

Sommaire

1. Conception de la base
2. Requêtes de jointures
3. Agrégations et calculs
4. Fonction SQL
5. IN / NOT IN / EXISTS
6. Optimisation : Index et Vues
7. DCL - Gestion des utilisateurs

1. Conception de la base

1.1 Création de la base

Création de la base de données **atelier_commerce** et sélection de celle-ci avec USE.

```
CREATE DATABASE atelier_commerce;
USE atelier_commerce;
```

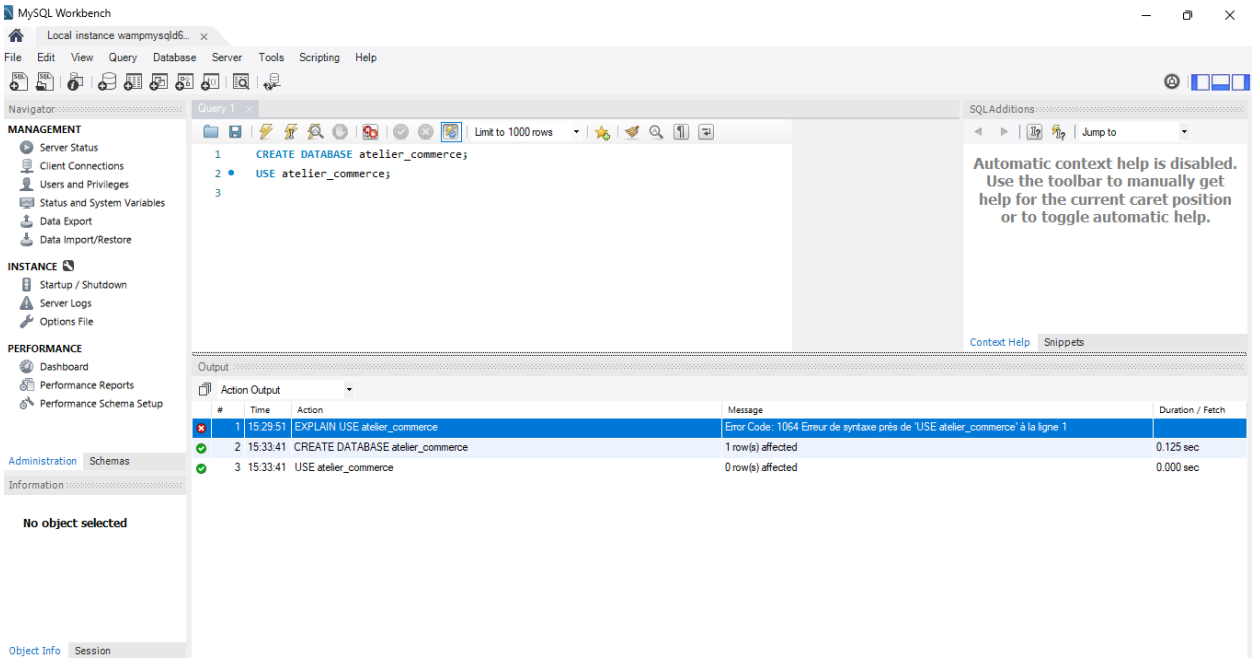


Figure 1 : Création de la base

1.2 Création des tables

Création des cinq tables : clients, produits, commandes, details_commandes et paiements, reliées par des clés étrangères.

```
CREATE TABLE clients (id INT AUTO_INCREMENT PRIMARY KEY, nom VARCHAR(50), ville VARCHAR(50));
CREATE TABLE produits (id INT AUTO_INCREMENT PRIMARY KEY, nom_produit VARCHAR(50), prix_unitaire DECIMAL(10,2));
CREATE TABLE commandes (id INT AUTO_INCREMENT PRIMARY KEY, client_id INT, date_commande DATE,
  FOREIGN KEY (client_id) REFERENCES clients(id));
CREATE TABLE details_commandes (id INT AUTO_INCREMENT PRIMARY KEY, commande_id INT, produit_id INT, quantite INT,
  FOREIGN KEY (commande_id) REFERENCES commandes(id), FOREIGN KEY (produit_id) REFERENCES produits(id));
CREATE TABLE paiements (id INT AUTO_INCREMENT PRIMARY KEY, commande_id INT, montant DECIMAL(10,2),
  mode_paiement VARCHAR(30), date_paiement DATE, FOREIGN KEY (commande_id) REFERENCES commandes(id));
```

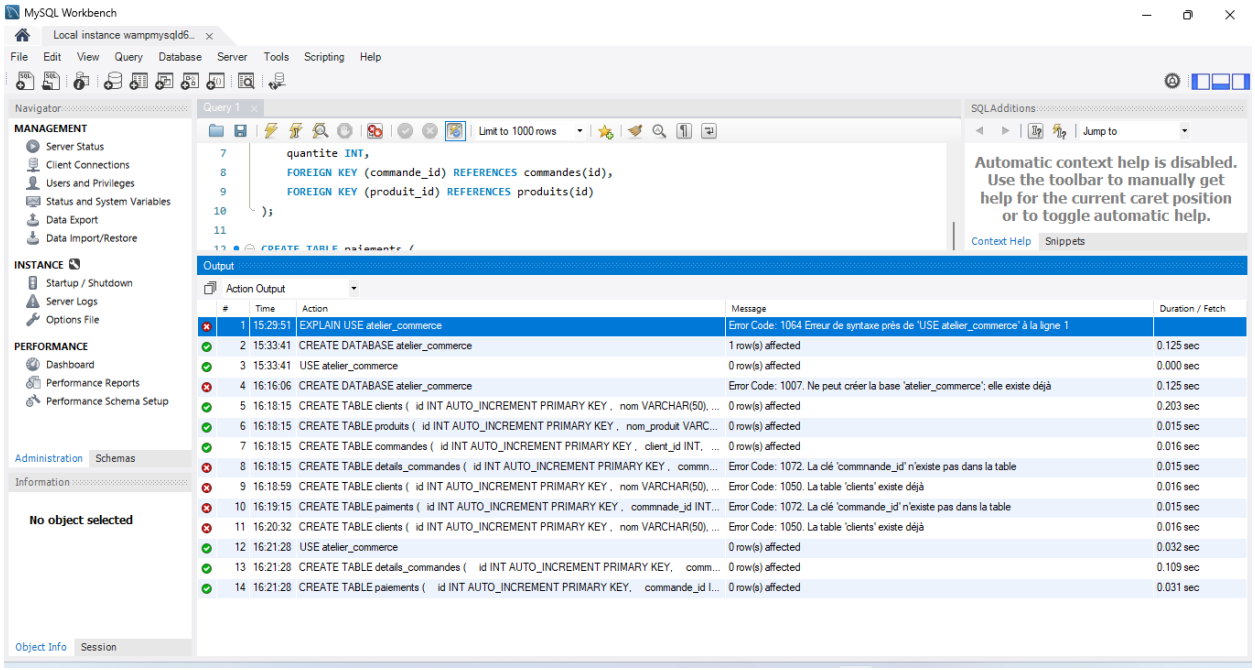


Figure 2 : Création des tables

1.3 Insertion des données

Insertion de données de test dans les tables.

```
INSERT INTO clients (nom,ville) VALUES ('Alice','paris'),('Bob','Lyon'),('Charlie','Toulouse'),('Dina','Nice');
INSERT INTO produits (nom_produit,prix_unitaire) VALUES ('ordinateur',950),('souris',20),('clavier',45),('ecran',2
-- commandes, details_commandes et paiements : même principe
```

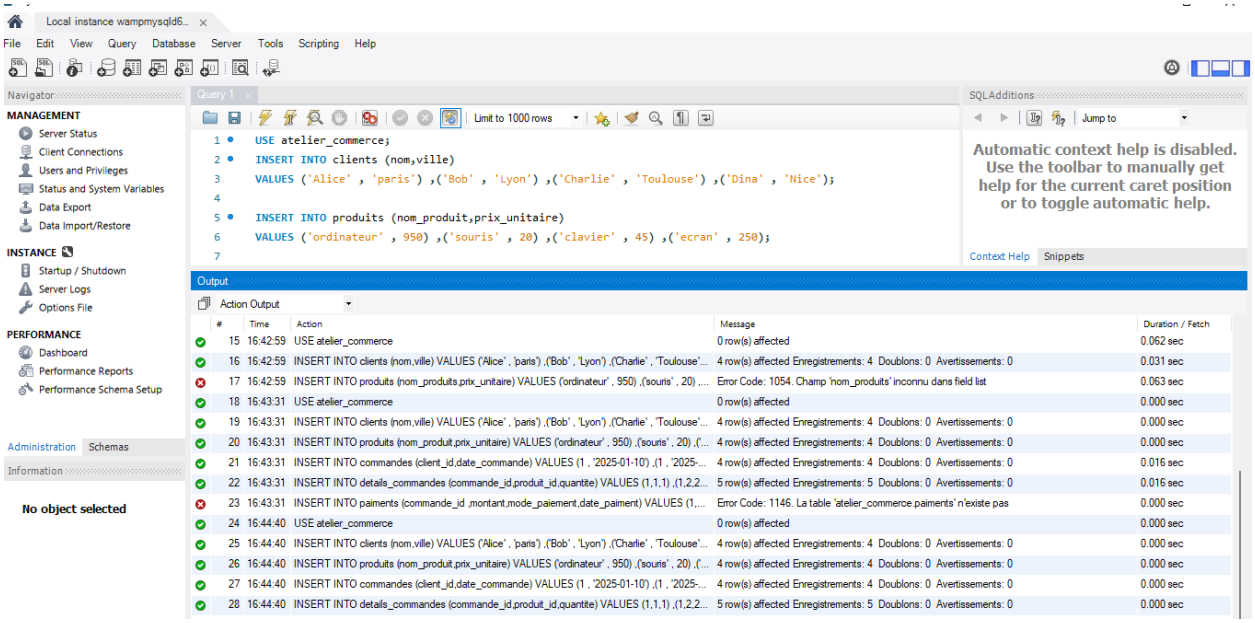


Figure 3 : Insertion des données

2. Requêtes de jointures

2.1 INNER JOIN

Affiche uniquement les lignes qui ont une correspondance dans toutes les tables (client, commande, produit, quantité).

```
SELECT c.nom, cmd.id AS commande, p.nom_produit, dc.quantite
FROM clients c
INNER JOIN commandes cmd ON c.id = cmd.client_id
INNER JOIN details_commandes dc ON cmd.id = dc.commande_id
INNER JOIN produits p ON dc.produit_id = p.id;
```

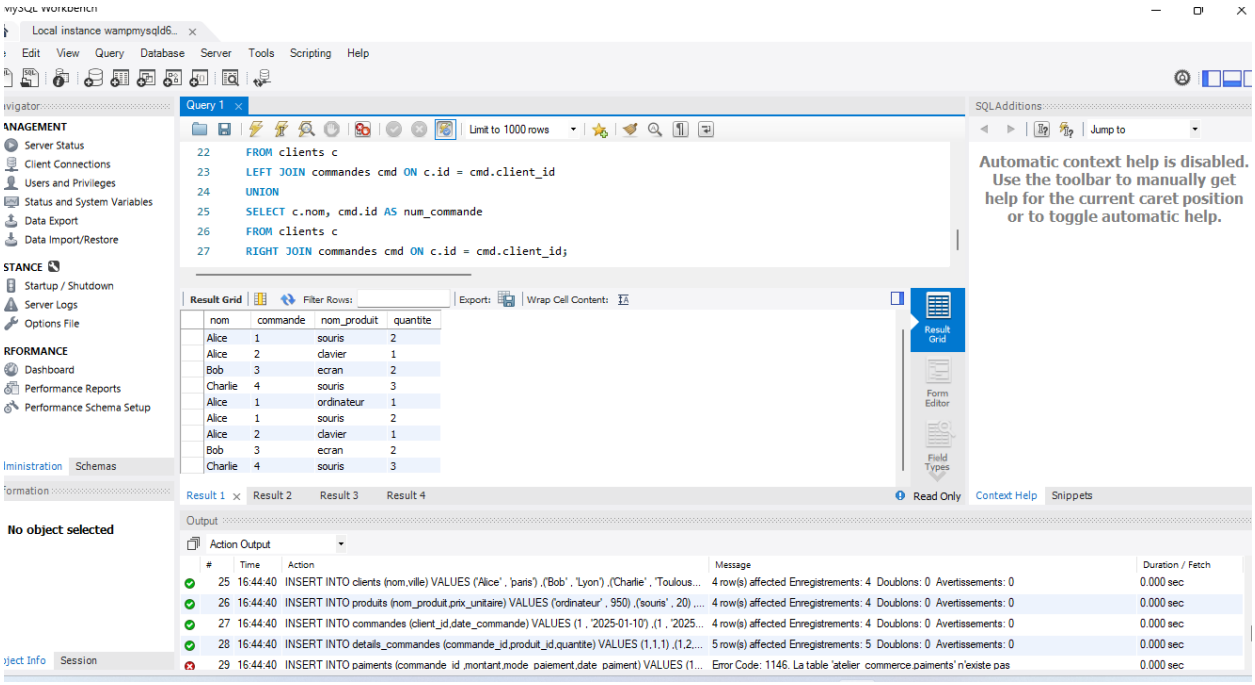


Figure 4 : INNER JOIN

2.2 LEFT JOIN

Retourne tous les clients, avec leurs commandes s'il y en a (NULL sinon).

```
SELECT c.nom, cmd.id AS num_commande
FROM clients c
LEFT JOIN commandes cmd ON c.id = cmd.client_id;
```

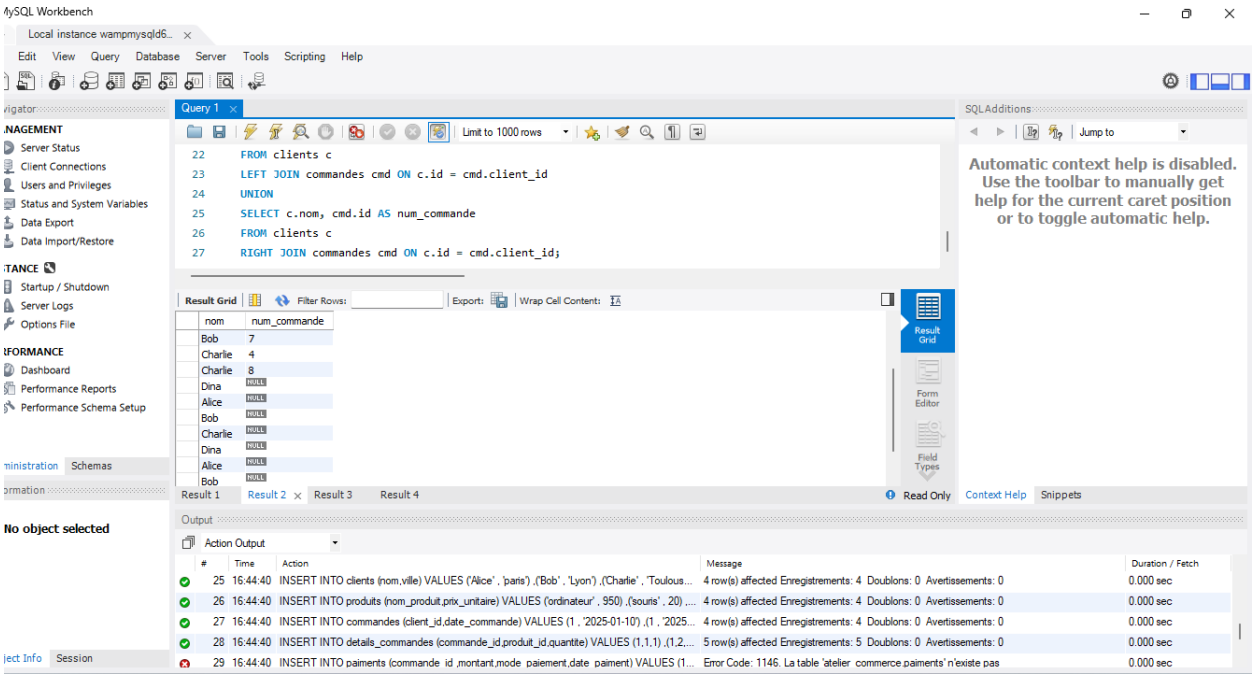


Figure 5 : LEFT JOIN

2.3 RIGHT JOIN

Retourne toutes les commandes, avec le client associé.

```
SELECT c.nom, cmd.id, cmd.date_commande
FROM clients c
RIGHT JOIN commandes cmd ON c.id = cmd.client_id;
```

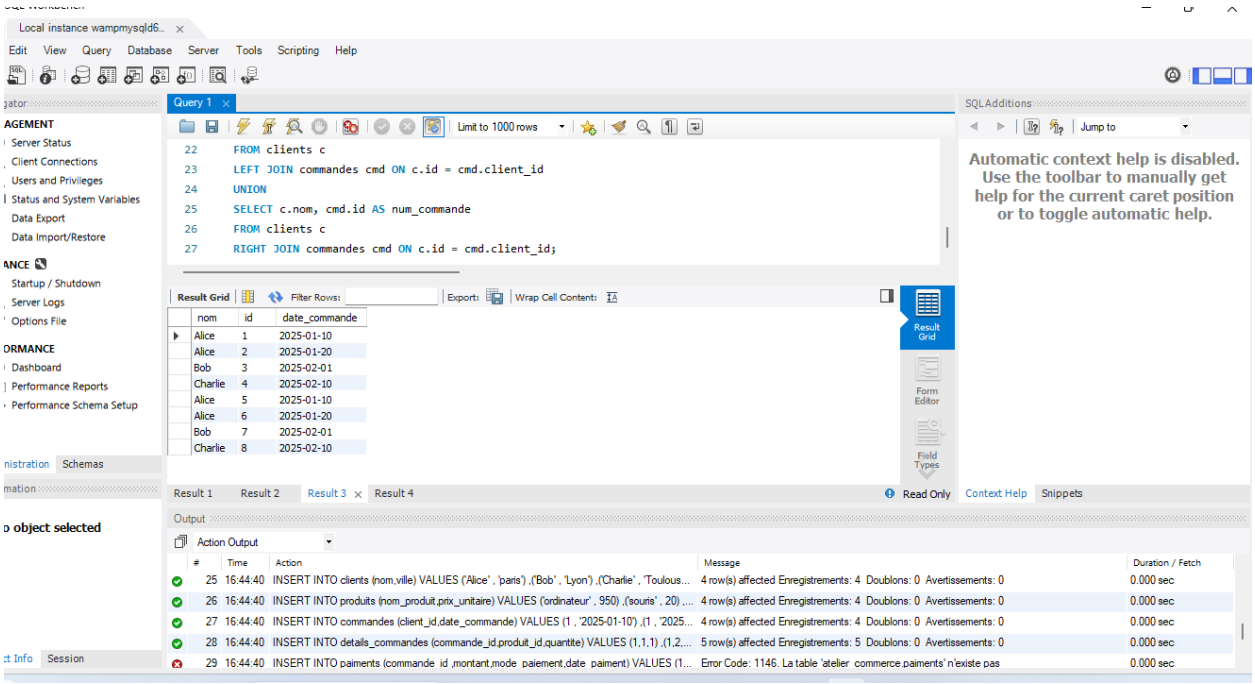


Figure 6 : RIGHT JOIN

2.4 FULL JOIN (simulation MySQL)

MySQL ne possède pas FULL JOIN : on combine LEFT JOIN et RIGHT JOIN avec UNION.

```
SELECT c.nom, cmd.id AS num_commande FROM clients c
LEFT JOIN commandes cmd ON c.id = cmd.client_id
UNION
SELECT c.nom, cmd.id AS num_commande FROM clients c
RIGHT JOIN commandes cmd ON c.id = cmd.client_id;
```

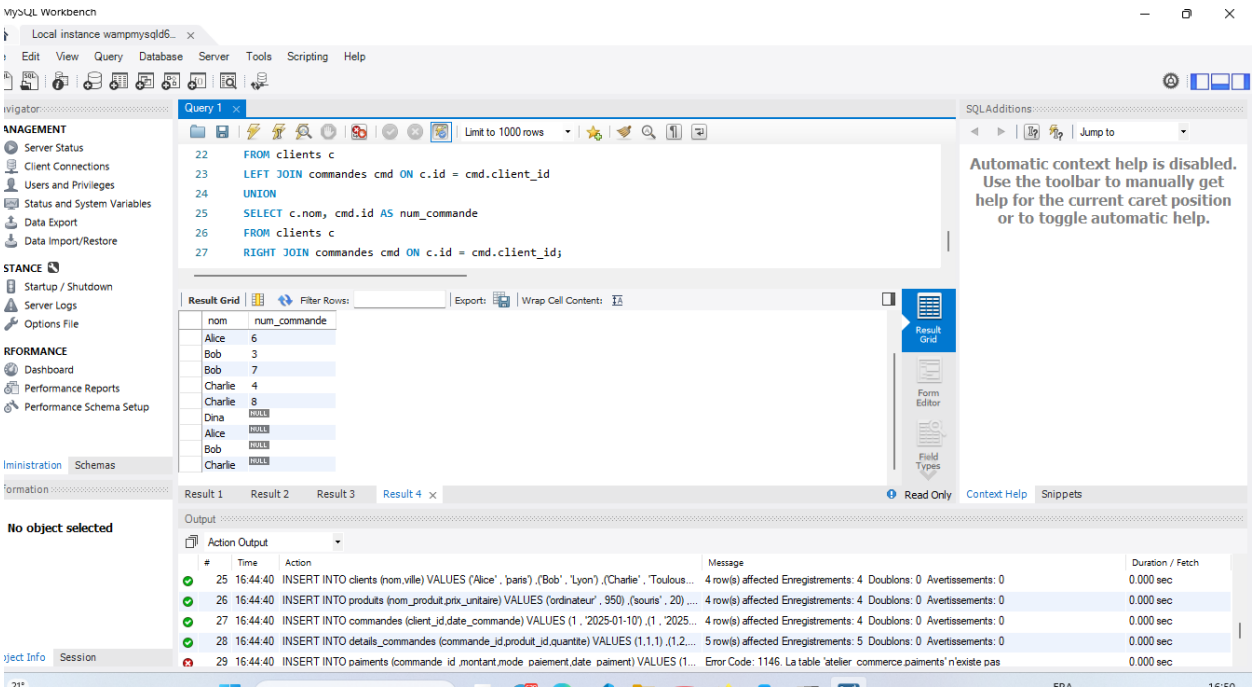


Figure 7 : FULL JOIN

3. Agrégations et calculs

3.1 Montant total par commande

Somme de prix_unitaire x quantite, regroupée par commande.

```
SELECT cmd.id AS commande, SUM(p.prix_unitaire * dc.quantite) AS total_commande
FROM commandes cmd
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
GROUP BY cmd.id;
```

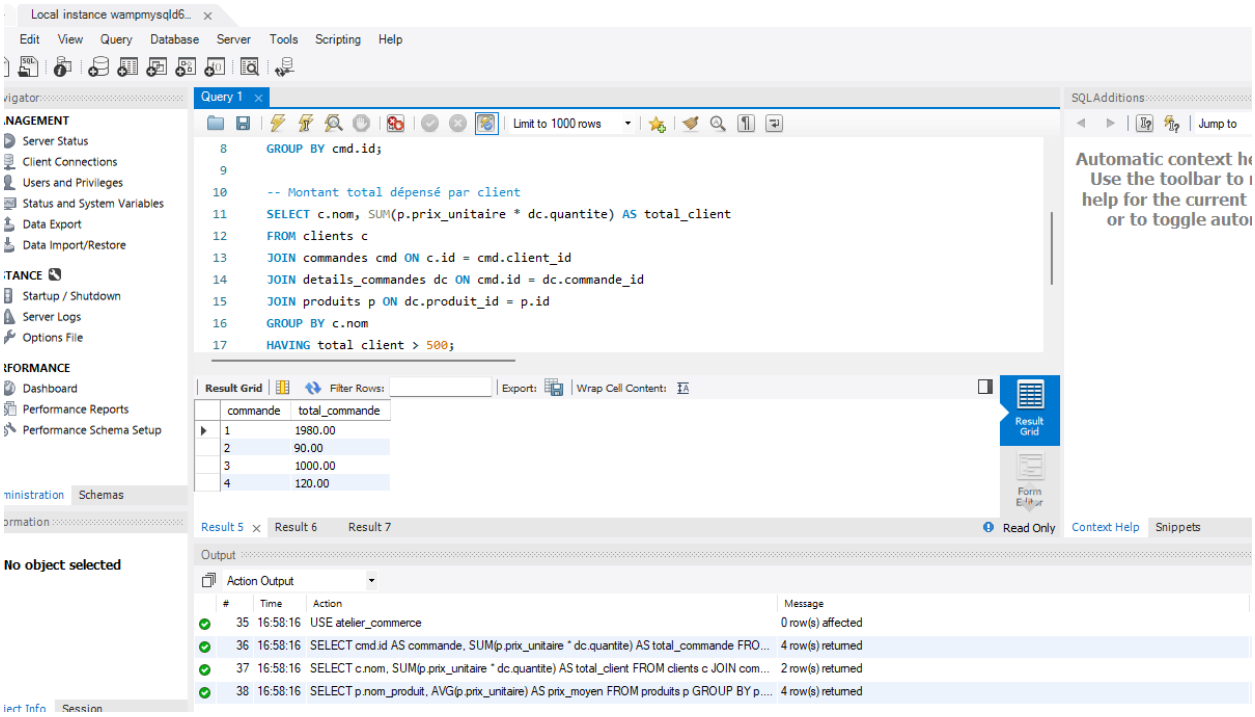


Figure 8 : Montant total par commande

3.2 Montant total dépensé par client

Total par client, filtré avec HAVING (total supérieur à 500).

```
SELECT c.nom, SUM(p.prix_unitaire * dc.quantite) AS total_client
FROM clients c
JOIN commandes cmd ON c.id = cmd.client_id
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
GROUP BY c.nom
HAVING total_client > 500;
```

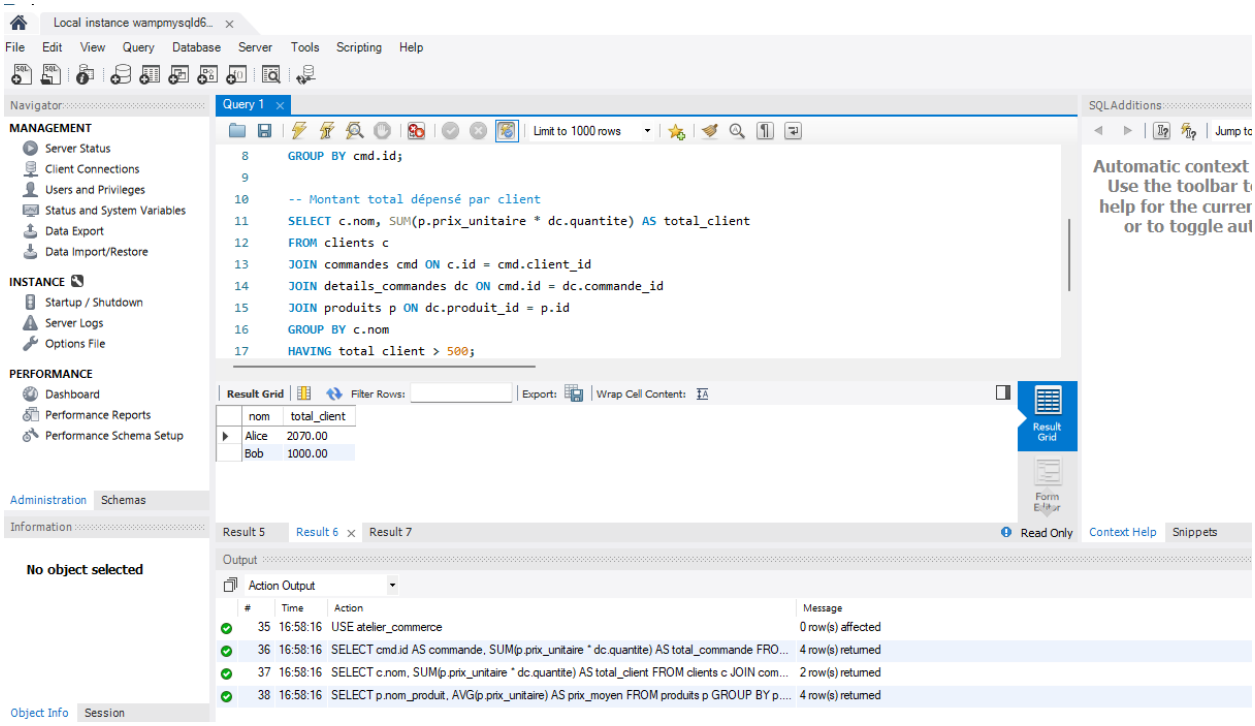


Figure 9 : Montant dépensé par client

3.3 Montant moyen par produit

Prix moyen par produit avec AVG et GROUP BY.

```
SELECT p.nom_produit, AVG(p.prix_unitaire) AS prix_moyen
FROM produits p
GROUP BY p.nom_produit;
```

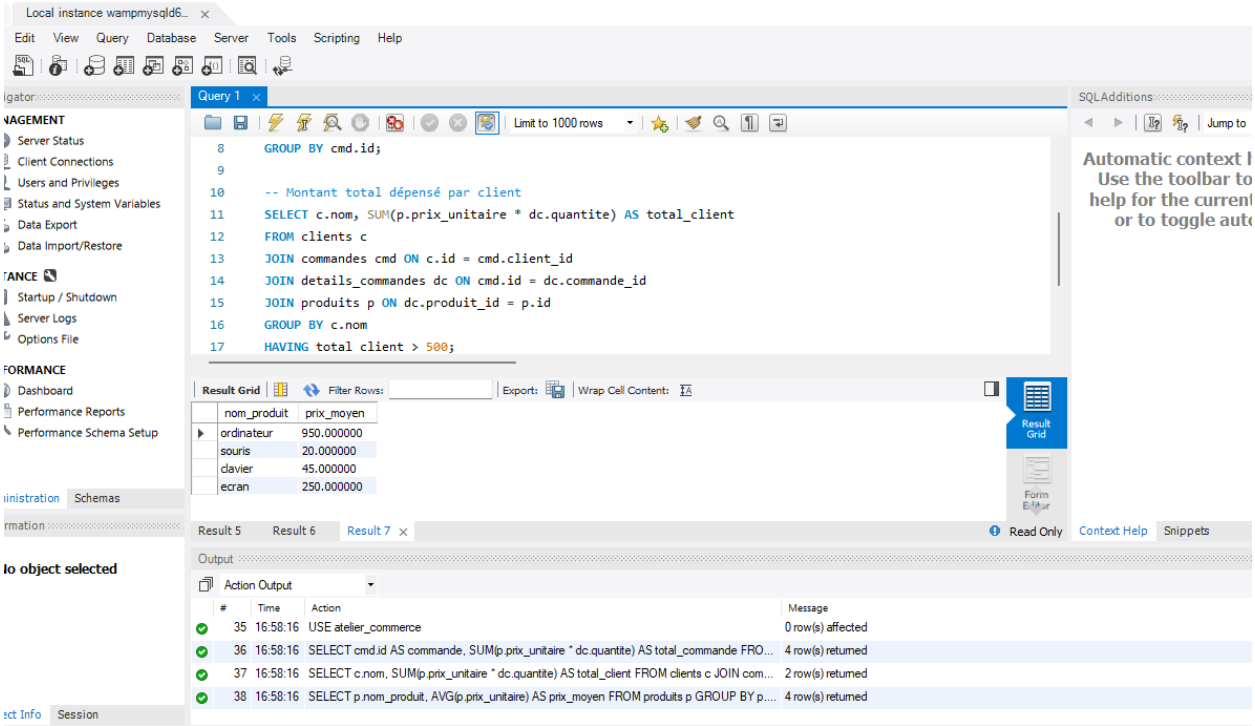


Figure 10 : Montant moyen par produit

4. Fonction SQL

4.1 IF() pour classer les clients

Classe chaque client en Premium (total supérieur à 800) ou Standard.

```
SELECT c.nom, SUM(p.prix_unitaire * dc.quantite) AS total,
  IF(SUM(p.prix_unitaire * dc.quantite) > 800, 'Premium', 'Standard') AS categorie
FROM clients c
JOIN commandes cmd ON c.id = cmd.client_id
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
GROUP BY c.nom;
```

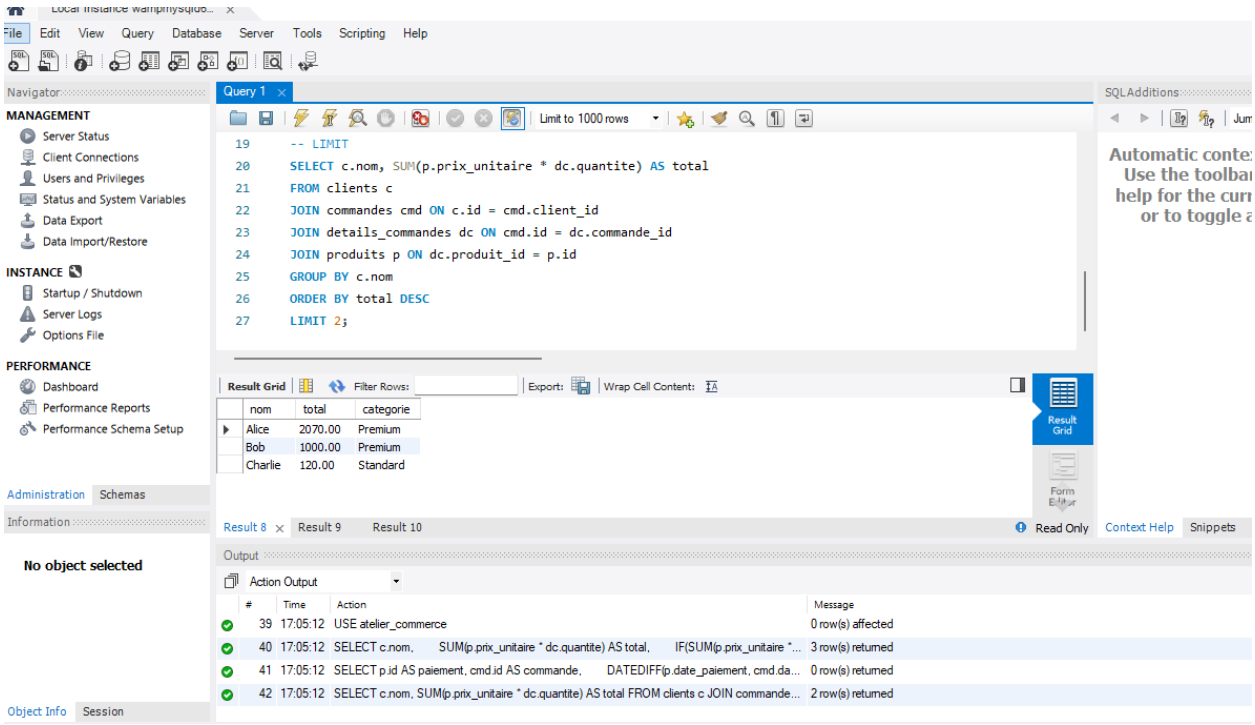


Figure 11 : Fonction IF()

4.2 Fonctions de date

Calcule le délai en jours entre la commande et le paiement avec DATEDIFF.

```
SELECT p.id AS paiement, cmd.id AS commande,
  DATEDIFF(p.date_paiement, cmd.date_commande) AS delai_jours
FROM paiements p
JOIN commandes cmd ON p.commande_id = cmd.id;
```

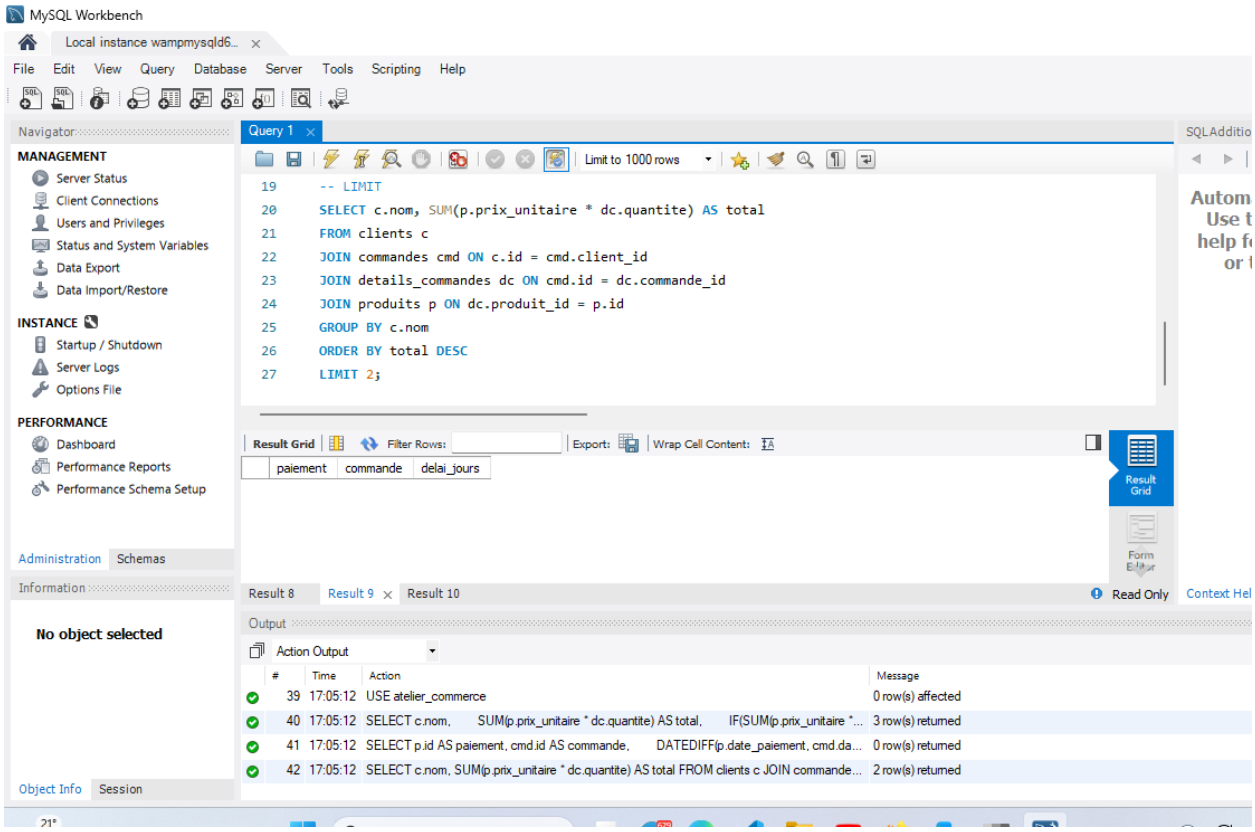


Figure 12 : Fonctions de date

4.3 LIMIT

Affiche les 2 clients ayant dépensé le plus (ORDER BY ... DESC + LIMIT 2).

```
SELECT c.nom, SUM(p.prix_unitaire * dc.quantite) AS total
FROM clients c
JOIN commandes cmd ON c.id = cmd.client_id
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
GROUP BY c.nom ORDER BY total DESC LIMIT 2;
```

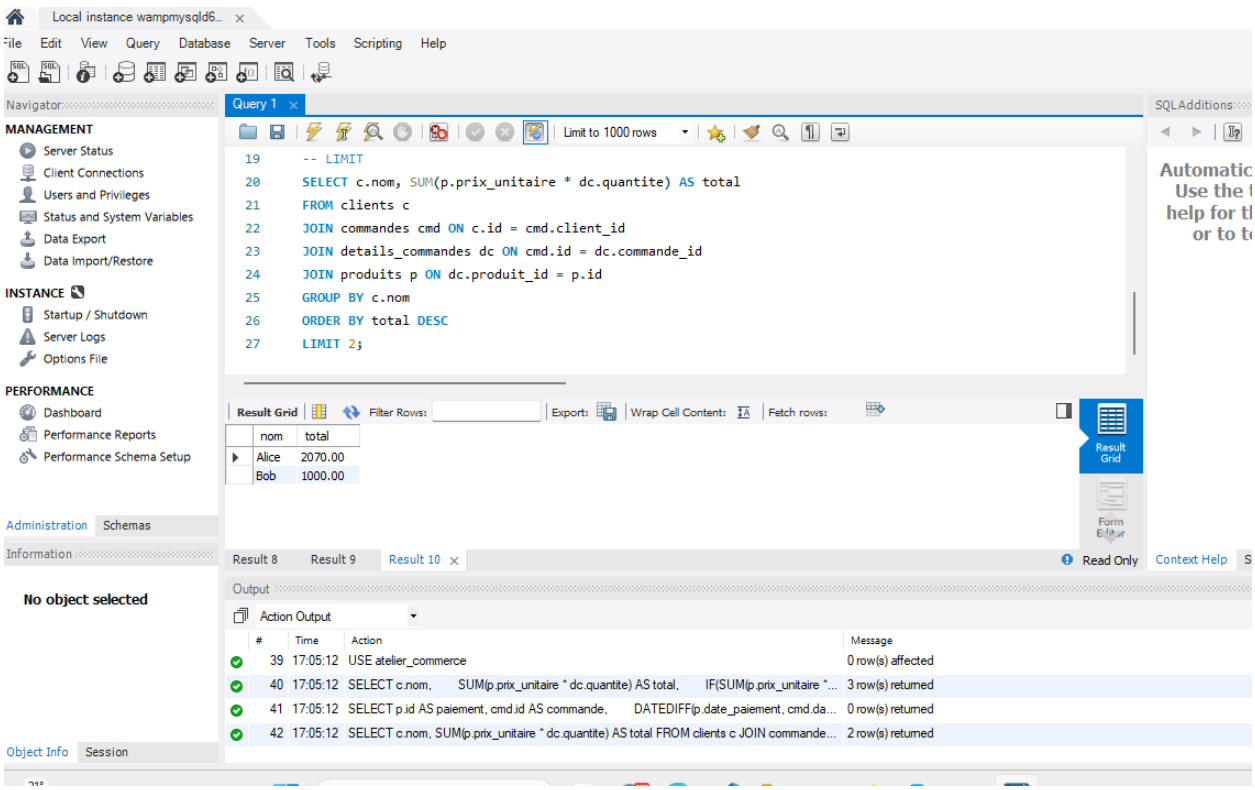


Figure 13 : LIMIT

5. IN / NOT IN / EXISTS

5.1 IN

Clients ayant au moins une commande payée.

```
SELECT nom FROM clients WHERE id IN (
  SELECT client_id FROM commandes WHERE id IN (
    SELECT commande_id FROM paiements));
```

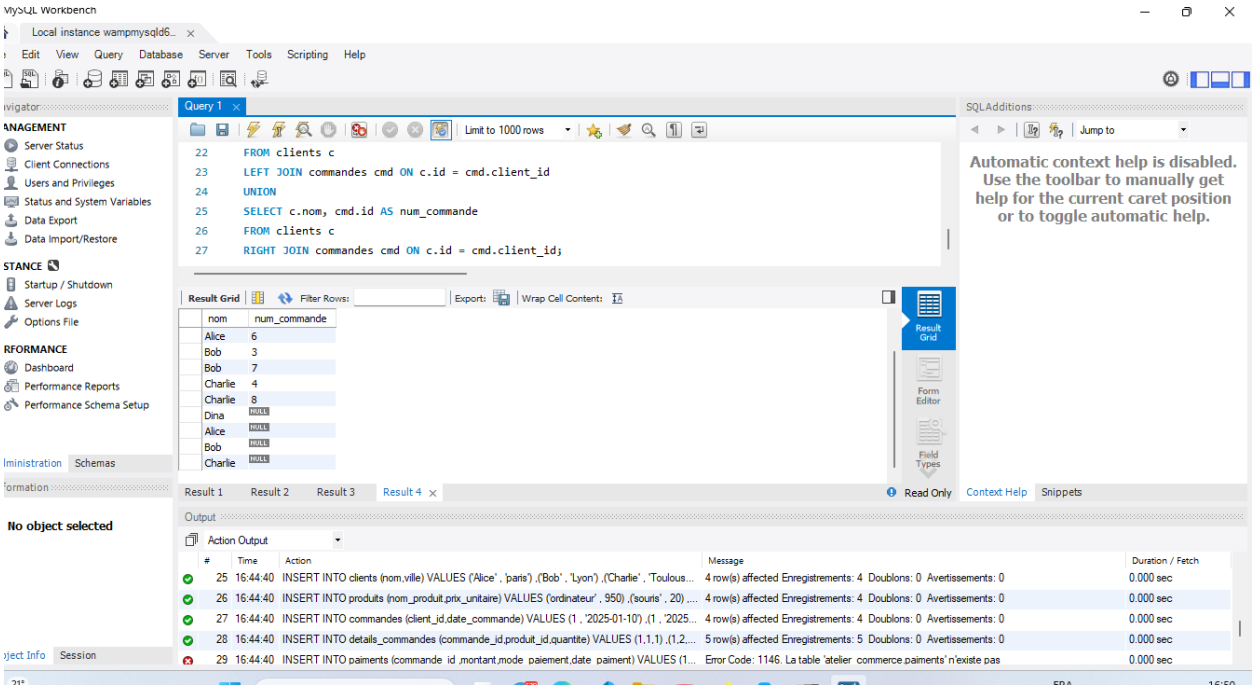


Figure 14 : IN

5.2 NOT IN

Clients n'ayant aucune commande payée.

```
SELECT nom FROM clients WHERE id NOT IN (
  SELECT client_id FROM commandes WHERE id IN (
    SELECT commande_id FROM paiements));
```

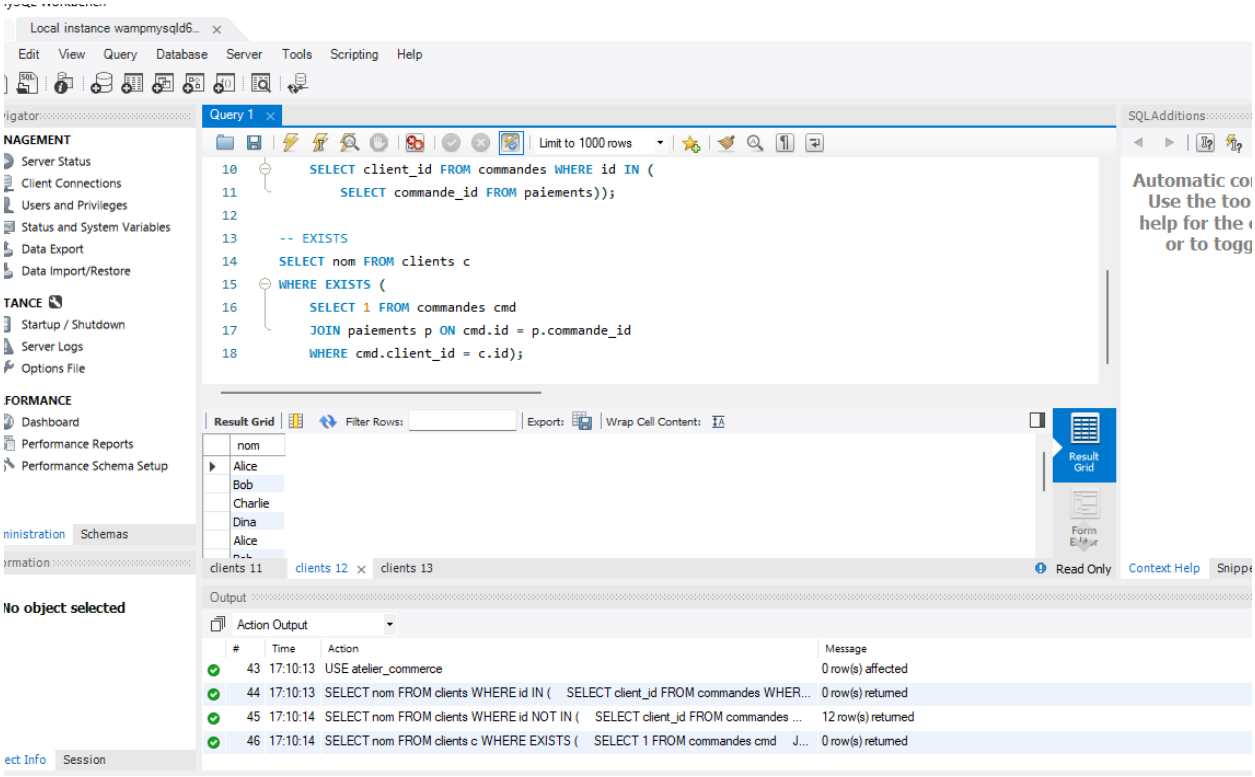


Figure 15 : NOT IN

5.3 EXISTS

Clients pour lesquels il existe une commande payée.

```
SELECT nom FROM clients c
WHERE EXISTS (
  SELECT 1 FROM commandes cmd
  JOIN paiements p ON cmd.id = p.commande_id
  WHERE cmd.client_id = c.id);
```

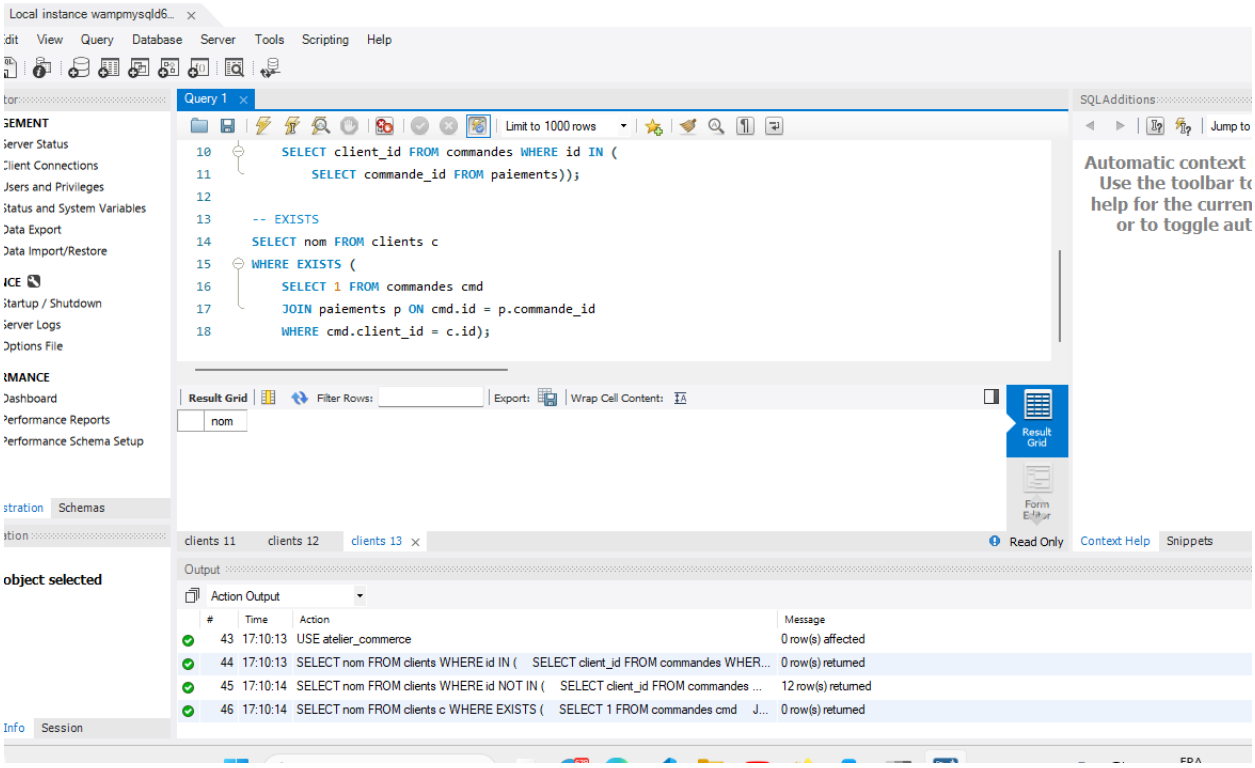


Figure 16 : EXISTS

6. Optimisation : Index et Vues

6.1 Index et EXPLAIN

Création d'index pour accélérer les recherches, puis EXPLAIN pour vérifier que l'index idx_nom_produit est utilisé (colonne key).

```
CREATE INDEX idx_nom_produit ON produits(nom_produit);
CREATE INDEX idx_client_ville ON clients(ville);
EXPLAIN SELECT * FROM produits WHERE nom_produit = 'Souris';
```

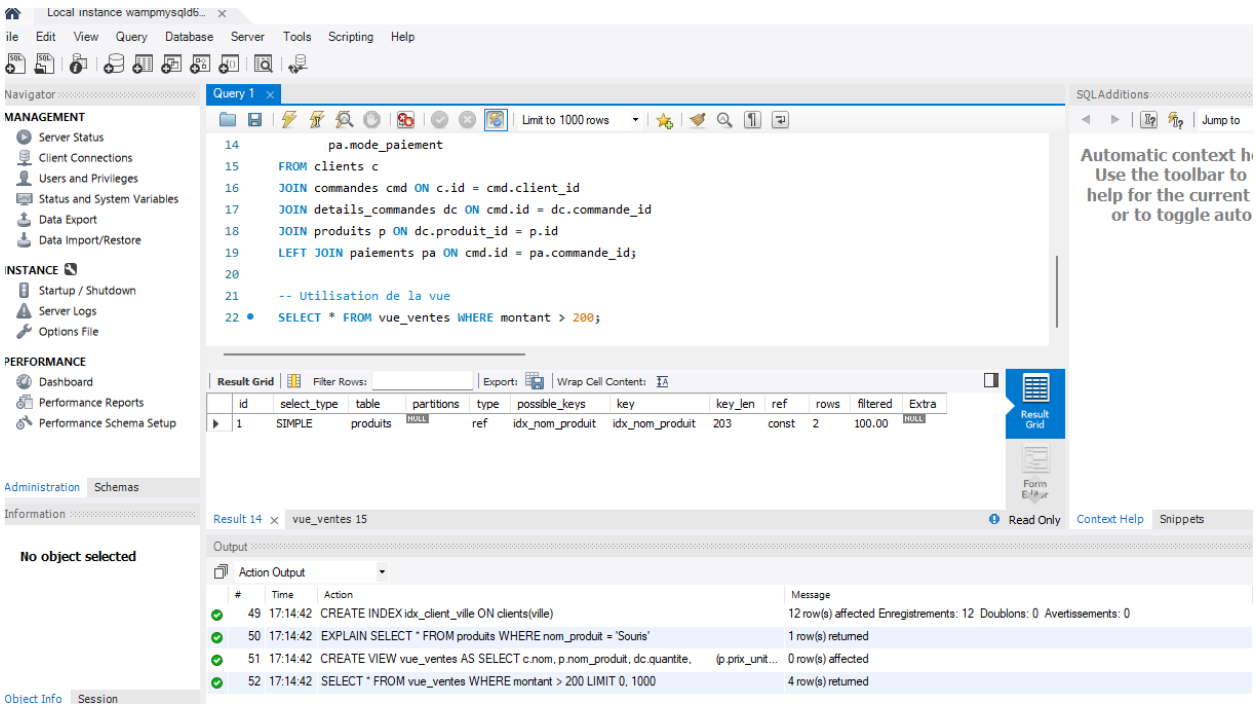


Figure 17 : Index et EXPLAIN

6.2 Vue vue_ventes

Création d'une vue regroupant client, produit, quantité, montant et mode de paiement, puis interrogation de la vue.

```
CREATE VIEW vue_ventes AS
SELECT c.nom, p.nom_produit, dc.quantite,
  (p.prix_unitaire * dc.quantite) AS montant, pa.mode_paiement
FROM clients c
JOIN commandes cmd ON c.id = cmd.client_id
JOIN details_commandes dc ON cmd.id = dc.commande_id
JOIN produits p ON dc.produit_id = p.id
LEFT JOIN paiements pa ON cmd.id = pa.commande_id;

SELECT * FROM vue_ventes WHERE montant > 200;
```

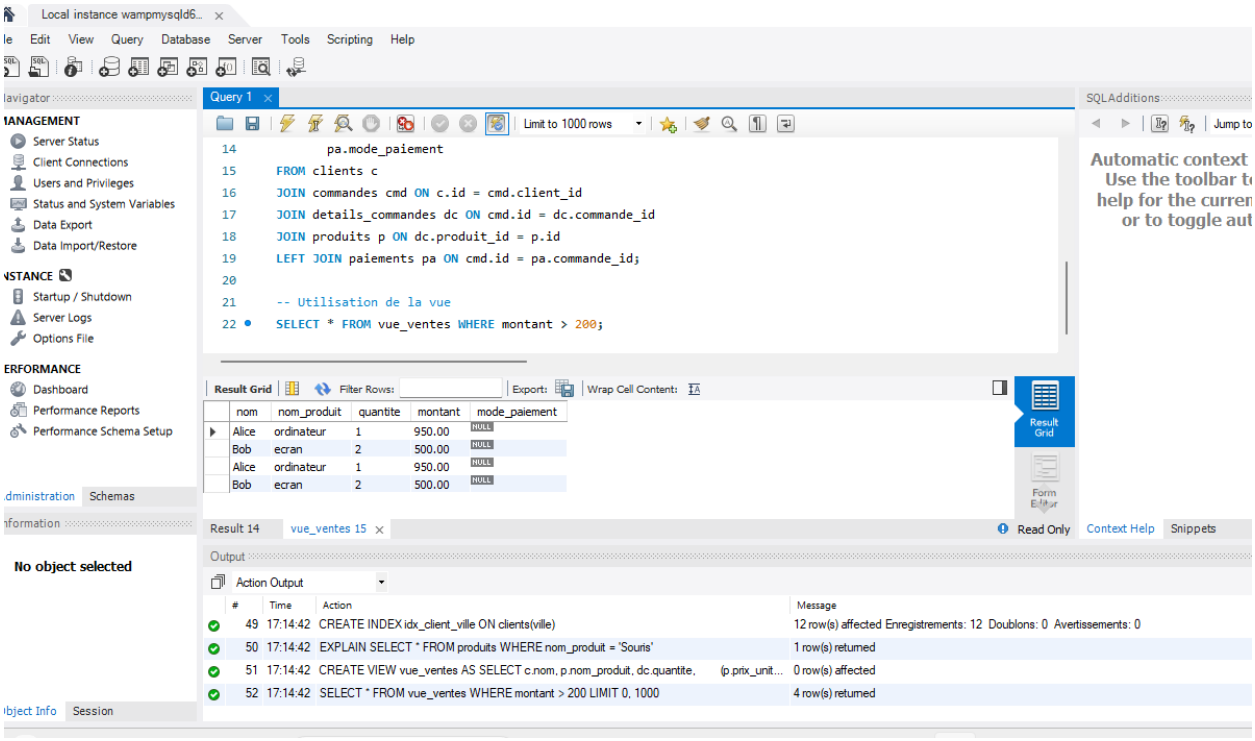


Figure 18 : Vue vue_ventes

7. DCL - Gestion des utilisateurs

7.1 Utilisateurs et privilèges

Création des utilisateurs vendeur et manager, attribution des droits avec GRANT, vérification avec SHOW GRANTS, puis retrait du droit UPDATE avec REVOKE.

```
CREATE USER 'vendeur'@'localhost' IDENTIFIED BY '1234';
CREATE USER 'manager'@'localhost' IDENTIFIED BY 'admin';
GRANT SELECT, INSERT, UPDATE ON atelier_commerce.* TO 'vendeur'@'localhost';
GRANT ALL PRIVILEGES ON atelier_commerce.* TO 'manager'@'localhost';
FLUSH PRIVILEGES;
SHOW GRANTS FOR 'manager'@'localhost';
REVOKE UPDATE ON atelier_commerce.* FROM 'vendeur'@'localhost';
```

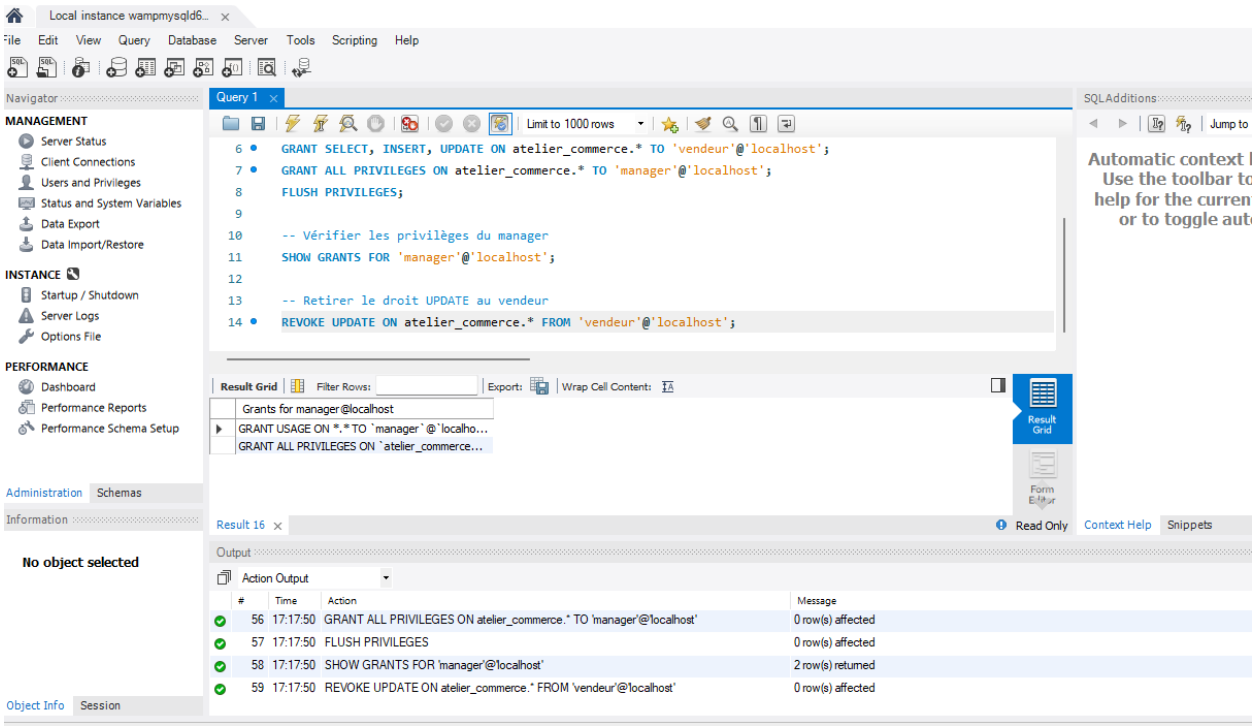


Figure 19 : Gestion des utilisateurs (DCL)